Project Introduction

In modern software systems, creating complex objects that contain multiple optional attributes can lead to complicated and hard-to-maintain code when using traditional constructors or setter methods. This project demonstrates how the Builder Design Pattern can be used to solve this problem by providing a flexible and readable way to construct complex objects step by step

.

The project simulates a simple meal ordering scenario where a user can create a customized Meal by selecting different components such as the main dish, drink, and side item. Since these components are optional and can vary between meals, the system requires a scalable and maintainable approach to object creation

.

To address this challenge, the Builder Pattern is implemented to separate the construction of the Meal object from its representation. This approach improves code readability, prevents the creation of incomplete objects, and makes the system easier to extend in the future

.

The project includes UML diagrams, implementation code, and documentation that demonstrate how the Builder Pattern is applied in practice following clean design principles

.

Pattern Selection & Justification

Pattern: Builder Design Pattern

لماذا مناسب؟

- الكائن يحتوي خصائص كثيرة/اختيارية
- إنشاء الكائن تدريجيًا
- كود أوضح وقابل للتوسعة

كيف حل المشكلة؟

بدل constructor طويل، يتم البناء خطوة بخطوة باستخدام Builder

الفوائد:

- إنشاء منظم وواضح
- منع كائنات غير مكتملة
- سهولة القراءة
- إضافة خصائص لاحقًا بسهولة

مبادئ التصميم:

- Single Responsibility
- Open/Closed
- Encapsulation

```
} public class Main

// الكلاس الأساسي (Immutable-style)
} static class Meal
;private final String mainDish
;private final String drink
;private final String side

} private Meal(Builder builder)
;this.mainDish = builder.mainDish
;this.drink = builder.drink
;this.side = builder.side
{
```

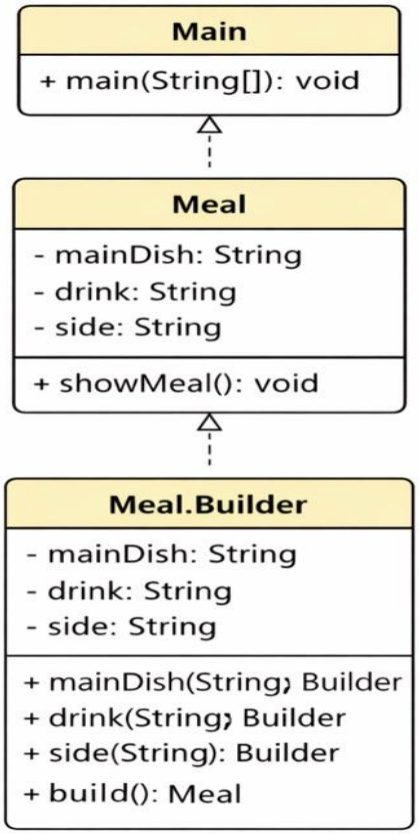
```
} ()public void showMeal
;System.out.println("Main Dish: " + mainDish)
;System.out.println("Drink: " + drink)
;System.out.println("Side: " + side)
{

// Builder الداخلي
} public static class Builder
;private String mainDish
;private String drink
;private String side

} public Builder mainDish(String mainDish)
;this.mainDish = mainDish
;return this
{

} public Builder drink(String drink)
;this.drink = drink
;return this
{

} public Builder side(String side)
;this.side = side
;return this
{
```

```
} ()public Meal build
;return new Meal(this)
{
{
{

// التشغيل
} public static void main(String[] args)
()Meal meal = new Meal.Builder
mainDish("Burger").
drink("Cola").
side("Fries").
;()build.

;()meal.showMeal
{
```

Builder Pattern UML Diagram Explanation

The UML diagram illustrates how the Builder Pattern is applied in the project. The Meal class represents the final product that contains the meal attributes such as main dish, drink, and side. The inner Builder class is responsible for constructing the object step by step by setting the required values, and then calling the build() method to create the final Meal object.

The diagram also shows the role of the Main class as the client, which uses the Builder to create different meals in a flexible and safe way. This approach prevents the creation of incomplete objects and improves scalability and maintainability.



Conclusion

In conclusion, the Builder Design Pattern provides a clean and flexible way to construct complex objects step by step. It improves code readability, prevents the creation of incomplete objects, and makes.

the system easier to maintain and extend. By applying this pattern, the design becomes more organized and aligned with key software engineering principles.

هبه كمال مدالله المبيضين..202300174

تسنيم ايمن توفيق العاروري 202301122

نور انس درويش الرجبي 202304807

.